Modul 5 PPBL



Muhammad Haikal
707012400003
Dosen pengampu: Mohammad As'ad Rosyadi, S. Pd., M.T.

Telkom University
Sistem Informasi Kota Cerdas
SIKC 48-03
2026-2027

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan perbedaan konsep antara *Provider*, *Cubit*, dan *BLoC* dalam pengelolaan *state* pada aplikasi Flutter.
- Mengimplementasikan *Cubit* menggunakan paket *flutter_bloc* untuk mengelola *state* sederhana tanpa *event* eksplisit.
- Mengimplementasikan *BLoC* dengan *Event* dan *State* untuk mengelola *state* secara terstruktur dan mudah diuji.
- Menghubungkan *Cubit* dan *Bloc* ke antarmuka pengguna menggunakan *BlocProvider*, *BlocBuilder*, *BlocListener*, dan *BlocConsumer*.
- Membandingkan kompleksitas, jumlah *boilerplate*, dan kemudahan penggunaan antara *Provider*, *Cubit*, dan *BLoC* pada studi kasus yang sama.
- Menentukan pendekatan *state* management yang tepat (*Provider*, *Cubit*, atau *BLoC*) berdasarkan kebutuhan dan skala aplikasi.
- Menggabungkan beberapa pendekatan *state* management dalam satu aplikasi multi-halaman dengan navigasi.

2. Alat dan Bahan

Perangkat Keras (Hardware):

- Laptop atau PC dengan prosesor setara Intel Core i3 atau lebih tinggi.
- Kapasitas memori RAM minimal 8 GB (direkomendasikan 16 GB untuk menjalankan Android Emulator secara optimal).
- Penyimpanan berbasis SSD dengan ruang kosong minimal 16 GB hingga 32 GB.
- Koneksi internet yang stabil untuk mengunduh dependensi dan paket pengaturan.

Perangkat Lunak (Software):

- Sistem operasi Windows, macOS, atau Linux.
- Flutter SDK (versi *stable*, contoh: v3.47.0) yang di dalamnya sudah terintegrasi langsung dengan Dart SDK (contoh: v3.13.0).
- Visual Studio Code sebagai penyunting kode utama, yang telah dilengkapi dengan ekstensi Flutter dan Dart.
- Android Studio, digunakan secara khusus untuk menyediakan Android SDK, Android SDK Command-line Tools, Android SDK Platform-Tools, serta mengelola perangkat keras virtual / Android Emulator.
- Git (contoh: v2.55.0 pada Windows) yang difungsikan sebagai sistem kendali versi dan merupakan prasyarat internal untuk menjalankan mesin Flutter.
- Peramban Google Chrome yang digunakan sebagai target platform saat menjalankan (*debugging*) aplikasi dalam format web.

3. Langkah-Langkah Praktikum

Bagian ini membangun ulang aplikasi Kantin Kampus dari Modul 3 sebanyak dua kali,: proyek pertama menggunakan *Cubit*, proyek kedua menggunakan *BLoC*. Kedua proyek meniru persis tampilan dan alur Modul 3 (halaman *Produk* dan halaman Keranjang dengan navigasi bawah), hanya cara pengelolaan *state*-nya yang berbeda. Untuk versi *Provider*, proyek Modul 3

yang sudah ada dipakai sebagai pembanding. Setiap proyek diselesaikan dan diuji sampai berjalan sempurna sebelum melanjutkan ke proyek berikutnya.

a. Menyiapkan Proyek Praktikum

Sebelum memulai, pastikan proyek Kantin Kampus dari Modul 3 (versi *Provider*) masih tersimpan dan dapat dijalankan, karena proyek tersebut akan dipakai sebagai pembanding pada Bagian g. Buat proyek baru untuk versi *Cubit* dengan menjalankan perintah pada Kode 5.4.

```
flutter create kantin_kampus_cubit
cd kantin_kampus_cubit
flutter pub add flutter_bloc
```

- Di dalam proyek kantin_kampus_*cubit*, buat tiga folder di dalam lib/: models/, *cubit*/, dan pages/

b. Membuat *CartCubit* untuk Mengelola Keranjang Belanja

Bekerja pada proyek kantin_kampus_*cubit*. Kelas *Produk* dan *ItemKeranjang* pada Modul 3 dipakai apa adanya (hanya kelas *KeranjangModel* yang tidak dipakai, karena logic-nya dipindahkan ke *KeranjangCubit*).

- Buat file lib/models/*produk*.dart, lalu salin program di bawah ini.

```
import 'package:flutter/foundation.dart';

@immutable
class Produk {
 const Produk({
  required this.nama,
  required this.harga,
  required this.ikon,
 });

 final String nama;
 final int harga;
 final String ikon;
}

class ItemKeranjang {
 const ItemKeranjang({required this.produk, this.jumlah = 1});

 final Produk produk;
 final int jumlah;
}
```

- Buat file lib/*cubit*/keranjang_*cubit*.dart, lalu tulis kode dibawah ini. *KeranjangCubit* menyimpan *state* berupa List<*ItemKeranjang*>, menggantikan Map<String,

ItemKeranjang> pada *KeranjangModel*, tetapi dengan hasil akhir yang sama: satu *produk* yang sama hanya muncul sekali di keranjang, dengan jumlah yang bertambah.

lib/cubit/keranjang_cubit.dart

```
import 'package:flutter_bloc/flutter_bloc.dart';

import '../models/produk.dart';

class KeranjangCubit extends Cubit<List<ItemKeranjang>> {
 KeranjangCubit() : super([]);

 void tambah(Produk produk) {
  final daftar = List<ItemKeranjang>.from(state);
  final indeks = daftar.indexWhere((item) => item.produk.nama == produk.nama);

  if (indeks == -1) {
   daftar.add(ItemKeranjang(produk: produk));
  } else {
   daftar[indeks] = ItemKeranjang(
    produk: produk,
    jumlah: daftar[indeks].jumlah + 1,
   );
  }

  emit(daftar);
 }

 void kurangi(Produk produk) {
  final daftar = List<ItemKeranjang>.from(state);
  final indeks = daftar.indexWhere((item) => item.produk.nama == produk.nama);
  if (indeks == -1) return;

  if (daftar[indeks].jumlah == 1) {
   daftar.removeAt(indeks);
  } else {
   daftar[indeks] = ItemKeranjang(
    produk: produk,
    jumlah: daftar[indeks].jumlah - 1,
   );
  }

  emit(daftar);
 }

 void kosongkan() {
  emit([]);
```

```
  }
}
```

- Perhatikan bahwa method tambah(), kurangi(), dan kosongkan() melakukan hal yang persis sama dengan method sejenis pada *KeranjangModel* Modul 3 (mencari *produk* yang sama, menambah/mengurangi jumlah, atau menghapus dari daftar). Perbedaannya hanya pada baris terakhir: *KeranjangModel* memanggil *notifyListeners*() setelah mengubah data secara langsung, sedangkan *KeranjangCubit* membuat daftar baru lalu mengirimkannya dengan *emit*().

c. Menghubungkan *Cubit* dengan *BlocProvider* dan *BlocBuilder*

Halaman *HalamanProduk* dan *HalamanKeranjang* dibuat mengikuti tampilan Modul 3 persis, hanya sumber datanya diganti dari *KeranjangModel* menjadi *KeranjangCubit*.

- Buat file lib/pages/halaman_*produk*.dart, lalu tulis kode dibawah ini. Bandingkan dengan lib/pages/halaman_*produk*.dart pada Modul 3, hanya baris context.read<*KeranjangModel*>() yang berubah menjadi context.read<*KeranjangCubit*>().

lib/pages/halaman_produk.dart (versi Cubit)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/keranjang_cubit.dart';
import '../models/produk.dart';

class HalamanProduk extends StatelessWidget {
 const HalamanProduk({super.key});

 static const daftarProduk = [
  Produk(nama: 'Nasi Goreng', harga: 15000, ikon: ' 🍛 '),
  Produk(nama: 'Mie Ayam', harga: 13000, ikon: ' 🍜 '),
  Produk(nama: 'Es Teh', harga: 5000, ikon: ' 🧋 '),
 ];

 String rupiah(int nilai) => 'Rp${nilai.toString()}';

 @override
 Widget build(BuildContext context) {
  return ListView.separated(
   padding: const EdgeInsets.all(16),
   itemCount: daftarProduk.length,
   separatorBuilder: (context, indeks) => const SizedBox(height: 8),
   itemBuilder: (context, indeks) {
    final produk = daftarProduk[indeks];
```

```
      return Card(
        child: ListTile(
          leading: Text(produk.ikon, style: const TextStyle(fontSize: 32)),
          title: Text(produk.nama),
          subtitle: Text(rupiah(produk.harga)),
          trailing: FilledButton(
            onPressed: () {
              context.read<KeranjangCubit>().tambah(produk);
            },
            child: const Text('Tambah'),
          ),
        ),
      );
    },
   );
  }
}
```

- Buat file lib/pages/halaman_keranjang.dart, lalu tulis kode dibawah ini. Bandingkan dengan lib/pages/halaman_keranjang.dart pada Modul 3 — *Consumer<KeranjangModel>* diganti menjadi *BlocBuilder<KeranjangCubit*, List<*ItemKeranjang*>>, dan keranjang.items dibaca langsung sebagai parameter items pada builder.

lib/pages/halaman_keranjang.dart (versi Cubit)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/keranjang_cubit.dart';
import '../models/produk.dart';

class HalamanKeranjang extends StatelessWidget {
 const HalamanKeranjang({super.key});

 String rupiah(int nilai) => 'Rp${nilai.toString()}';

 @override
 Widget build(BuildContext context) {
  return BlocBuilder<KeranjangCubit, List<ItemKeranjang>>(
   builder: (context, items) {
    if (items.isEmpty) {
     return const Center(
      child: Column(
       mainAxisSize: MainAxisSize.min,
       children: [
        Icon(Icons.shopping_cart_outlined, size: 64),
```

```
        SizedBox(height: 12),
        Text('Keranjang masih kosong'),
      ],
    ),
  );
}

final totalHarga = items.fold<int>(
  0,
  (total, item) => total + (item.produk.harga * item.jumlah),
);

return Column(
  children: [
    Expanded(
      child: ListView.separated(
        padding: const EdgeInsets.all(16),
        itemCount: items.length,
        separatorBuilder: (context, indeks) => const Divider(),
        itemBuilder: (context, indeks) {
          final item = items[indeks];

          return ListTile(
            title: Text(item.produk.nama),
            subtitle: Text('${item.jumlah} x ${rupiah(item.produk.harga)}'),
            trailing: Row(
              mainAxisSize: MainAxisSize.min,
              children: [
                IconButton(
                  onPressed: () =>
                      context.read<KeranjangCubit>().kurangi(item.produk),
                  icon: const Icon(Icons.remove_circle_outline),
                ),
                Text('${item.jumlah}'),
                IconButton(
                  onPressed: () =>
                      context.read<KeranjangCubit>().tambah(item.produk),
                  icon: const Icon(Icons.add_circle_outline),
                ),
              ],
            ),
          );
        },
      ),
    ),
    SafeArea(
      top: false,
```

```
          child: Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Text(
                  'Total: ${rupiah(totalHarga)}',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const SizedBox(height: 8),
                OutlinedButton.icon(
                  onPressed: () => context.read<KeranjangCubit>().kosongkan(),
                  icon: const Icon(Icons.delete_outline),
                  label: const Text('Kosongkan Keranjang'),
                ),
              ],
            ),
          ),
        ),
      ],
    );
  },
 );
 }
}
```

- Ganti seluruh isi lib/main.dart. Struktur ini identik dengan lib/main.dart pada Modul 3 (*KerangkaUtama* dengan *NavigationBar* antara *Produk* dan Keranjang), hanya *ChangeNotifierProvider* diganti *BlocProvider* dan *KeranjangModel* diganti *KeranjangCubit*.

lib/main.dart (proyek kantin_kampus_cubit)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'cubit/keranjang_cubit.dart';
import 'pages/halaman_keranjang.dart';
import 'pages/halaman_produk.dart';

void main() {
 runApp(
  BlocProvider(
   create: (context) => KeranjangCubit(),
   child: const AplikasiKantin(),
  ),
 );
```

```
}

class AplikasiKantin extends StatelessWidget {
 const AplikasiKantin({super.key});

 @override
 Widget build(BuildContext context) {
  return MaterialApp(
   debugShowCheckedModeBanner: false,
   title: 'Kantin Kampus (Cubit)',
   theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
    useMaterial3: true,
   ),
   home: const KerangkaUtama(),
  );
 }
}

class KerangkaUtama extends StatefulWidget {
 const KerangkaUtama({super.key});

 @override
 State<KerangkaUtama> createState() => _KerangkaUtamaState();
}

class _KerangkaUtamaState extends State<KerangkaUtama> {
 int _indeks = 0;

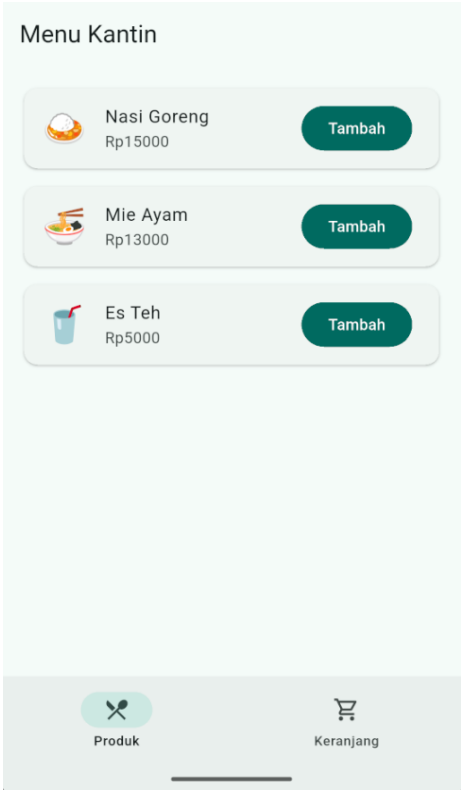
 static const _halaman = [
  HalamanProduk(),
  HalamanKeranjang(),
 ];

 @override
 Widget build(BuildContext context) {
  final totalItem = context.select<KeranjangCubit, int>(
   (cubit) => cubit.state.fold(0, (total, item) => total + item.jumlah),
  );

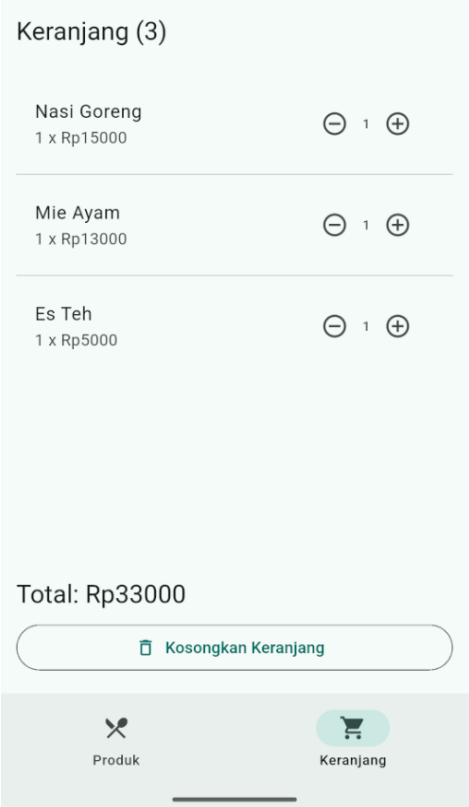
  return Scaffold(
   appBar: AppBar(
    title: Text(_indeks == 0 ? 'Menu Kantin' : 'Keranjang ($totalItem)'),
   ),
   body: IndexedStack(
    index: _indeks,
    children: _halaman,
```

```
      ),
      bottomNavigationBar: NavigationBar(
       selectedIndex: _indeks,
       onDestinationSelected: (nilaiBaru) {
        setState(() {
         _indeks = nilaiBaru;
        });
       },
       destinations: const [
        NavigationDestination(
         icon: Icon(Icons.restaurant_menu),
         label: 'Produk',
        ),
        NavigationDestination(
         icon: Icon(Icons.shopping_cart_outlined),
         selectedIcon: Icon(Icons.shopping_cart),
         label: 'Keranjang',
        ),
       ],
      ),
     );
   }
}
```

- Jalankan **flutter run**, lalu uji seluruh alur: tambah beberapa *produk* pada halaman *Produk*, pindah ke halaman Keranjang untuk melihat daftar dan total harga, kurangi/tambah jumlah item, dan coba tombol Kosongkan Keranjang. Pastikan semuanya berjalan sama seperti pada Modul 3 sebelum melanjutkan ke proyek *BLoC*.



Gambar 5.1 Tampilan halaman Produk pada proyek kantin_kampus_cubit



Gambar 5.2 Tampilan halaman Keranjang pada proyek kantin_kampus_cubit

d. Membuat *CartBloc* dengan *Event* dan *State*

- Buat proyek kedua untuk versi *BLoC*

```
flutter create kantin_kampus_bloc
cd kantin_kampus_bloc
flutter pub add flutter_bloc
```

- Di dalam proyek kantin_kampus_*bloc*, buat tiga folder di dalam lib/: models/, *bloc*/, dan pages/
- Buat file lib/models/*produk*.dart, lalu salin program di bawah ini.

```
import 'package:flutter/foundation.dart';

@immutable
class Produk {
  const Produk({
    required this.nama,
    required this.harga,
    required this.ikon,
  });

  final String nama;
  final int harga;
  final String ikon;
}

class ItemKeranjang {
  const ItemKeranjang({required this.produk, this.jumlah = 1});

  final Produk produk;
  final int jumlah;
}
```

- Buat file lib/*bloc*/keranjang_*bloc*.dart, lalu tulis kode berikut. Berbeda dengan *Cubit*, *BLoC* memerlukan dua kelas tambahan: *KeranjangEvent* (aksi yang dikirim dari tampilan) dan *KeranjangState* (data yang ditampilkan ke layar).

lib/bloc/keranjang_bloc.dart

```
import 'package:flutter_bloc/flutter_bloc.dart';

import '../models/produk.dart';

// Event: daftar aksi yang bisa dikirim ke KeranjangBloc
abstract class KeranjangEvent {}

class TambahProduk extends KeranjangEvent {
  TambahProduk(this.produk);
```

```
  final Produk produk;
}

class KurangiProduk extends KeranjangEvent {
  KurangiProduk(this.produk);
  final Produk produk;
}

class KosongkanKeranjang extends KeranjangEvent {}

// State: data yang ditampilkan ke layar
class KeranjangState {
  const KeranjangState(this.items);
  final List<ItemKeranjang> items;
}

// Bloc: menerima Event, lalu mengeluarkan State baru
class KeranjangBloc extends Bloc<KeranjangEvent,
KeranjangState> {
  KeranjangBloc() : super(const KeranjangState([])) {
    on<TambahProduk>((event, emit) {
      final daftar = List<ItemKeranjang>.from(state.items);
      final indeks = daftar.indexWhere((item) => item.produk.nama ==
event.produk.nama);

      if (indeks == -1) {
        daftar.add(ItemKeranjang(produk: event.produk));
      } else {
        daftar[indeks] = ItemKeranjang(
          produk: event.produk,
          jumlah: daftar[indeks].jumlah + 1,
        );
      }

      emit(KeranjangState(daftar));
    });

    on<KurangiProduk>((event, emit) {
      final daftar = List<ItemKeranjang>.from(state.items);
      final indeks = daftar.indexWhere((item) => item.produk.nama ==
event.produk.nama);
      if (indeks == -1) return;

      if (daftar[indeks].jumlah == 1) {
        daftar.removeAt(indeks);
      } else {
        daftar[indeks] = ItemKeranjang(
```

```
        produk: event.produk,
        jumlah: daftar[indeks].jumlah - 1,
      );
    }

    emit(KeranjangState(daftar));
  });

  on<KosongkanKeranjang>((event, emit) {
    emit(const KeranjangState([]));
  });
 }
}
```

- Perhatikan bahwa isi setiap handler (on<*TambahProduk*>, on<*KurangiProduk*>, on<*KosongkanKeranjang*>) persis sama dengan isi method tambah(), kurangi(), dan kosongkan() pada *KeranjangCubit*. Perbedaannya hanya pada cara aksi dikirim: *Cubit* memanggil method secara langsung, sedangkan *BLoC* harus melalui *Event*.

e. Menghubungkan *Bloc* dengan *BlocProvider* dan *BlocBuilder*

Halaman dibuat mengikuti struktur yang sama dengan proyek *Cubit*, hanya cara mengirim aksi dan tipe *state* yang berbeda.

- Buat file lib/pages/halaman_*produk*.dart, lalu tulis kode berikut.

lib/pages/halaman_produk.dart (versi BLoC)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

class HalamanProduk extends StatelessWidget {
 const HalamanProduk({super.key});

 static const daftarProduk = [
  Produk(nama: 'Nasi Goreng', harga: 15000, ikon: ' 🍛 '),
  Produk(nama: 'Mie Ayam', harga: 13000, ikon: ' 🍜 '),
  Produk(nama: 'Es Teh', harga: 5000, ikon: ' 🧃 '),
 ];

 String rupiah(int nilai) => 'Rp${nilai.toString()}';

 @override
 Widget build(BuildContext context) {
  return ListView.separated(
   padding: const EdgeInsets.all(16),
```

```
    itemCount: daftarProduk.length,
    separatorBuilder: (context, indeks) => const SizedBox(height: 8),
    itemBuilder: (context, indeks) {
     final produk = daftarProduk[indeks];

     return Card(
      child: ListTile(
       leading: Text(produk.ikon, style: const TextStyle(fontSize: 32)),
       title: Text(produk.nama),
       subtitle: Text(rupiah(produk.harga)),
       trailing: FilledButton(
        onPressed: () {

context.read<KeranjangBloc>().add(TambahProduk(produk));
        },
        child: const Text('Tambah'),
       ),
      ),
     );
    },
   );
 }
}
```

- Buat file lib/pages/halaman_keranjang.dart, lalu tulis kode berikut.

lib/pages/halaman_keranjang.dart (versi BLoC)

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

class HalamanKeranjang extends StatelessWidget {
 const HalamanKeranjang({super.key});

 String rupiah(int nilai) => 'Rp${nilai.toString()}';

 @override
 Widget build(BuildContext context) {
  return BlocBuilder<KeranjangBloc, KeranjangState>(
   builder: (context, state) {
    if (state.items.isEmpty) {
     return const Center(
      child: Column(
       mainAxisSize: MainAxisSize.min,
       children: [
```

```
            Icon(Icons.shopping_cart_outlined, size: 64),
            SizedBox(height: 12),
            Text('Keranjang masih kosong'),
          ],
        ),
      );
    }

    final totalHarga = state.items.fold<int>(
      0,
      (total, item) => total + (item.produk.harga * item.jumlah),
    );

    return Column(
      children: [
        Expanded(
          child: ListView.separated(
            padding: const EdgeInsets.all(16),
            itemCount: state.items.length,
            separatorBuilder: (context, indeks) => const Divider(),
            itemBuilder: (context, indeks) {
              final item = state.items[indeks];

              return ListTile(
                title: Text(item.produk.nama),
                subtitle: Text('${item.jumlah} x
${rupiah(item.produk.harga)}'),
                trailing: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    IconButton(
                      onPressed: () => context
                          .read<KeranjangBloc>()
                          .add(KurangiProduk(item.produk)),
                      icon: const Icon(Icons.remove_circle_outline),
                    ),
                    Text('${item.jumlah}'),
                    IconButton(
                      onPressed: () => context
                          .read<KeranjangBloc>()
                          .add(TambahProduk(item.produk)),
                      icon: const Icon(Icons.add_circle_outline),
                    ),
                  ],
                ),
              );
            },
```

```
          ),
        ),
        SafeArea(
          top: false,
          child: Padding(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                Text(
                  'Total: ${rupiah(totalHarga)}',
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                const SizedBox(height: 8),
                OutlinedButton.icon(
                  onPressed: () =>

context.read<KeranjangBloc>().add(KosongkanKeranjang()),
                  icon: const Icon(Icons.delete_outline),
                  label: const Text('Kosongkan Keranjang'),
                ),
              ],
            ),
          ),
        ),
      ],
    );
  },
 );
 }
}
```

- Ganti seluruh isi lib/main.dart.

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/keranjang_bloc.dart';
import 'pages/halaman_keranjang.dart';
import 'pages/halaman_produk.dart';

void main() {
 runApp(
  BlocProvider(
   create: (context) => KeranjangBloc(),
   child: const AplikasiKantin(),
  ),
```

```
  );
}

class AplikasiKantin extends StatelessWidget {
 const AplikasiKantin({super.key});

 @override
 Widget build(BuildContext context) {
  return MaterialApp(
   debugShowCheckedModeBanner: false,
   title: 'Kantin Kampus (BLoC)',
   theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
    useMaterial3: true,
   ),
   home: const KerangkaUtama(),
  );
 }
}

class KerangkaUtama extends StatefulWidget {
 const KerangkaUtama({super.key});

 @override
 State<KerangkaUtama> createState() => _KerangkaUtamaState();
}

class _KerangkaUtamaState extends State<KerangkaUtama> {
 int _indeks = 0;

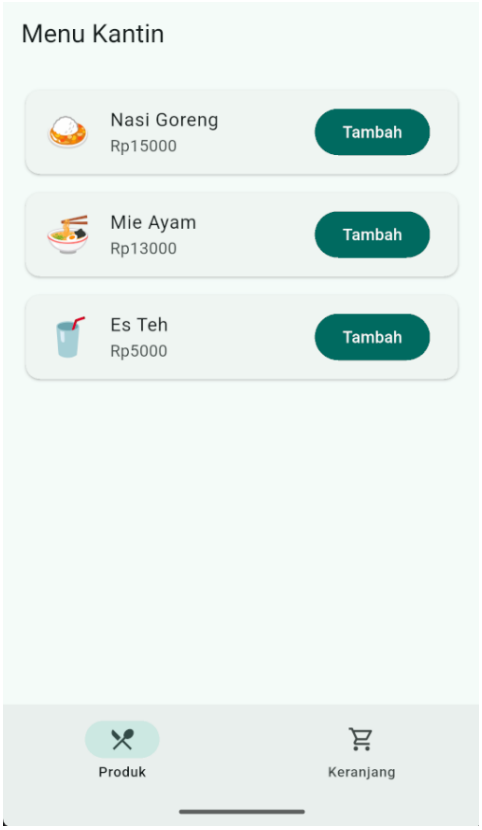
 static const _halaman = [
  HalamanProduk(),
  HalamanKeranjang(),
 ];

 @override
 Widget build(BuildContext context) {
  final totalItem = context.select<KeranjangBloc, int>(
   (bloc) => bloc.state.items.fold(0, (total, item) => total + item.jumlah),
  );

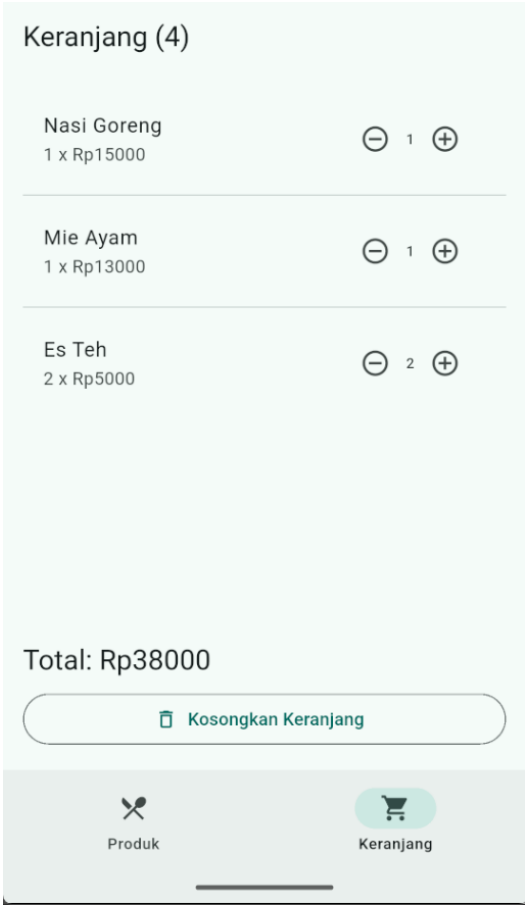
  return Scaffold(
   appBar: AppBar(
    title: Text(_indeks == 0 ? 'Menu Kantin' : 'Keranjang ($totalItem)'),
   ),
   body: IndexedStack(
    index: _indeks,
```

```
      children: _halaman,
    ),
    bottomNavigationBar: NavigationBar(
     selectedIndex: _indeks,
     onDestinationSelected: (nilaiBaru) {
      setState(() {
       _indeks = nilaiBaru;
      });
     },
     destinations: const [
      NavigationDestination(
       icon: Icon(Icons.restaurant_menu),
       label: 'Produk',
      ),
      NavigationDestination(
       icon: Icon(Icons.shopping_cart_outlined),
       selectedIcon: Icon(Icons.shopping_cart),
       label: 'Keranjang',
      ),
     ],
    ),
   );
  }
}
```

- Jalankan flutter run, lalu uji seluruh alur yang sama seperti pada proyek *Cubit*. Dari sisi tampilan, hasilnya identik dengan kantin_kampus_*cubit*; perbedaannya hanya pada cara mengirim aksi: *Cubit* memanggil method secara langsung, sedangkan *BLoC* selalu mengirim *Event* melalui add().



Gambar 5.3 Tampilan halaman Produk pada proyek kantin_kampus_bloc



Gambar 5.4 Tampilan halaman Keranjang pada proyek kantin_kampus_bloc

f. Menampilkan Efek Samping dengan *BlocListener* dan *BlocConsumer*

Sejauh ini, *BlocBuilder* dipakai untuk menampilkan data yang tersimpan pada *state*, misalnya daftar item di keranjang. Namun, ada jenis aksi lain yang sifatnya hanya perlu dijalankan satu kali ketika sebuah kejadian terjadi, bukan setiap kali tampilan digambar ulang, misalnya memunculkan SnackBar setiap kali *produk* berhasil ditambahkan. Aksi semacam ini disebut efek samping (*side effect*). Jika efek samping seperti ini diletakkan di dalam *BlocBuilder*, SnackBar berisiko muncul berulang-ulang setiap kali tampilan digambar ulang. Karena itu, *flutter_bloc* menyediakan dua *widget* khusus: *BlocListener* dan *BlocConsumer*.

- *BlocListener* hanya menjalankan efek samping (seperti menampilkan SnackBar) setiap kali *state* berubah, tanpa ikut menggambar *widget* apa pun ke layar.
- *BlocBuilder* hanya membangun ulang tampilan berdasarkan *state* terbaru, tidak cocok dipakai untuk efek samping.
- *BlocConsumer* menggabungkan keduanya dalam satu *widget*: parameter listener untuk efek samping, parameter builder untuk membangun tampilan.
- Kode di bawah adalah isi lengkap lib/pages/halaman_*produk*.dart pada proyek kantin_kampus_*bloc* setelah ditambahkan SnackBar menggunakan *BlocListener*, sehingga setiap kali tombol Tambah ditekan, muncul notifikasi singkat di bagian bawah layar.

lib/pages/halaman_produk.dart *dengan BlocListener*

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/keranjang_bloc.dart';
import '../models/produk.dart';

class HalamanProduk extends StatelessWidget {
 const HalamanProduk({super.key});

 static const daftarProduk = [
  Produk(nama: 'Nasi Goreng', harga: 15000, ikon: ' 🍛 '),
  Produk(nama: 'Mie Ayam', harga: 13000, ikon: ' 🍜 '),
  Produk(nama: 'Es Teh', harga: 5000, ikon: ' 🧃 '),
 ];

 String rupiah(int nilai) => 'Rp${nilai.toString()}';

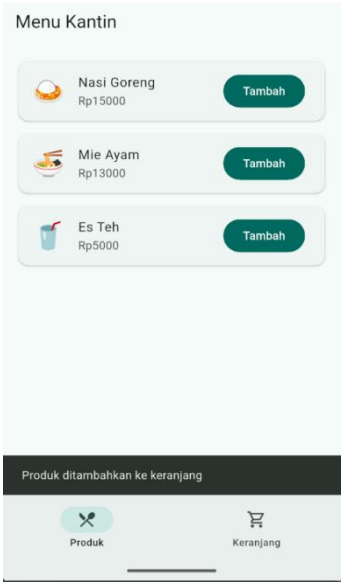
 @override
 Widget build(BuildContext context) {
  return BlocListener<KeranjangBloc, KeranjangState>(
   listener: (context, state) {
    if (state.items.isNotEmpty) {
```

```
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Produk ditambahkan ke
keranjang')),
        );
       }
     },
     child: ListView.separated(
       padding: const EdgeInsets.all(16),
       itemCount: daftarProduk.length,
       separatorBuilder: (context, indeks) => const SizedBox(height:
8),
       itemBuilder: (context, indeks) {
        final produk = daftarProduk[indeks];

        return Card(
         child: ListTile(
          leading: Text(produk.ikon, style: const TextStyle(fontSize:
32)),
          title: Text(produk.nama),
          subtitle: Text(rupiah(produk.harga)),
          trailing: FilledButton(
           onPressed: () {

context.read<KeranjangBloc>().add(TambahProduk(produk));
           },
           child: const Text('Tambah'),
          ),
         ),
        );
       },
     ),
   );
  }
}
```

- Perhatikan bahwa *BlocListener* membungkus (child:) tampilan ListView yang sudah ada, tanpa mengubah isi ListView itu sendiri. Listener hanya berjalan di latar belakang, menampilkan SnackBar setiap kali *state* berubah dan keranjang tidak kosong.
  - Pola yang sama juga berlaku untuk *Cubit* pada proyek kantin_kampus_*cubit*, karena *Cubit* dan *Bloc* sama-sama merupakan turunan dari kelas dasar *BlocBase* pada *flutter_bloc*.



Gambar 5.5 Tampilan SnackBar yang muncul setelah produk ditambahkan

g. Membandingkan *Provider*, *Cubit*, dan *BLoC* pada Layar yang Sama

Berikut merupakan rangkuman perbandingan konkret antara *Provider*, *Cubit*, dan *BLoC*, sebagaimana Tabel 5.2 berikut.

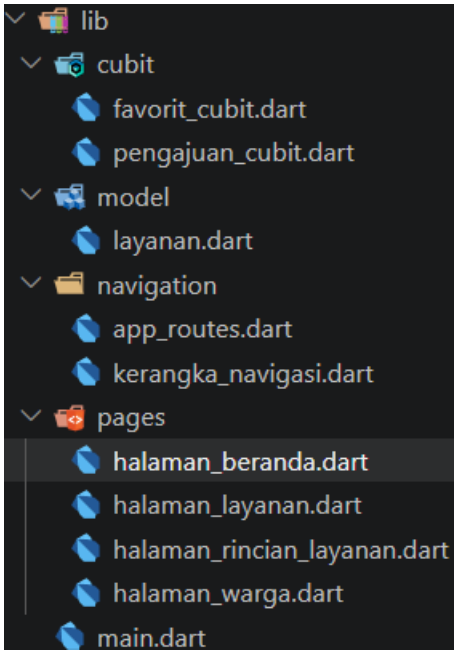
Tabel 5.2 Perbandingan Konkret Provider, Cubit, dan BLoC

Aspek	***Provider (Modul 3)***	***Cubit (kantin_kampus_cubit)***	***BLoC (kantin_kampus_bloc)***
Kelas dasar yang di-extends	*ChangeNotifier*	*Cubit<List<ItemKeranjang>>*	*Bloc<KeranjangEvent, KeranjangState>*
Bentuk data (state)	*Map<String, ItemKeranjang> di dalam kelas, dibaca lewat getter items*	*State langsung berupa List<ItemKeranjang>*	*State berupa objek KeranjangState yang membungkus List<ItemKeranjang> items*
Cara mengubah data	*Mengubah _items secara langsung, lalu notifyListeners()*	*Membuat daftar baru, lalu emit(daftarBaru)*	*Mengirim Event (mis. TambahProduk) dengan add(), ditangani handler on<...>, lalu emit(stateBaru)*
Cara widget membaca data	*Consumer<KeranjangModel>*	*BlocBuilder<KeranjangCubit, List<ItemKeranjang>>*	*BlocBuilder<KeranjangBloc, KeranjangState>*
Widget penyedia (root aplikasi)	*ChangeNotifierProvider*	*BlocProvider*	*BlocProvider*
Jumlah kelas yang harus dibuat	*1 kelas (KeranjangModel)*	*1 kelas (KeranjangCubit)*	*3 kelas (KeranjangEvent, KeranjangState, KeranjangBloc)*
Aksi dari tombol Tambah	*context.read<KeranjangModel>().tambah(produk)*	*context.read<KeranjangCubit>().tambah(produk)*	*context.read<KeranjangBloc>().add(TambahProduk(produk))*

4. Hasil Dokumentasi Program

4.1 Hasil Dokumentasi Program Nusantara Cerdas

a. Menyiapkan File lib (Halaman)



b. Halaman Beranda

```dart
import 'package:flutter/material.dart';

import '../navigation/app_routes.dart';

class HalamanBeranda extends StatelessWidget {
  const HalamanBeranda({super.key});

  @override
  Widget build(BuildContext context) {
    return ListView(
      padding: const EdgeInsets.all(20),
      children: [
        Card(
          child: Padding(
            padding: const EdgeInsets.all(24),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Icon(
                  Icons.account_balance,
                  size: 50,
                ), // Icon
                const SizedBox(height: 16),
                Text(
                  'Selamat Datang!',
                  style: Theme.of(context)
                      .textTheme
                      .headlineSmall
                      ?.copyWith(fontWeight: FontWeight.bold),
                ), // Text
                const SizedBox(height: 8),
                const Text(
                  'Nusantara Cerdas Mobile membantu warga '
                  'menemukan layanan pemerintah, menyimpan '
                  'layanan favorit, dan mengajukan beberapa '
                  'layanan dalam satu kunjungan.',
                ), // Text
                const SizedBox(height: 16),
                FilledButton.icon(
```

b. File Halaman layanan

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/favorit_cubit.dart';
import '../model/layanan.dart';
import '../navigation/app_routes.dart';

class HalamanLayanan extends StatelessWidget {
  const HalamanLayanan({super.key});

  Future<void> _bukaRincian(
    BuildContext context,
    Layanan layanan,
  ) async {
    final hasil = await Navigator.pushNamed(
      context,
      AppRoutes.rincianLayanan,
      arguments: layanan,
    );

    if (!context.mounted) return;

    if (hasil is String) {
      ScaffoldMessenger.of(context)
        ..hideCurrentSnackBar()
        ..showSnackBar(
          SnackBar(content: Text(hasil)),
        );
    }
  }
```

c. File Halaman rincian layanan

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/favorit_cubit.dart';
import '../cubit/pengajuan_cubit.dart';
import '../model/layanan.dart';

class HalamanRincianLayanan extends StatefulWidget {
  final Layanan layanan;

  const HalamanRincianLayanan({
    super.key,
    required this.layanan,
  });

  @override
  State<HalamanRincianLayanan> createState() =>
      _HalamanRincianLayananState();
}

class _HalamanRincianLayananState
    extends State<HalamanRincianLayanan> {
  bool _sedangMengirim = false;

  Future<void> _ajukanPermohonan() async {
    if (_sedangMengirim) return;

    setState(() {
      _sedangMengirim = true;
    });

    // Simulasi proses pengiriman permohonan.
    await Future<void>.delayed(
      const Duration(seconds: 1),
    ); // Future.delayed

    if (!mounted) return;

    final cubit = context.read<PengajuanCubit>();
```

d. File Halaman warga

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/favorit_cubit.dart';
import '../cubit/pengajuan_cubit.dart';
import '../model/layanan.dart';
import '../navigation/app_routes.dart';

class HalamanWarga extends StatelessWidget {
  const HalamanWarga({super.key});

  void _bukaLayanan(
    BuildContext context,
    String namaLayanan,
  ) {
    final layanan = KatalogLayanan.cari(namaLayanan);

    if (layanan == null) return;

    Navigator.pushNamed(
      context,
      AppRoutes.rincianLayanan,
      arguments: layanan,
    );
  }

  void _kirimSemua(BuildContext context) {
    final cubit = context.read<PengajuanCubit>();
    final jumlah = cubit.state.totalPengajuan;

    if (jumlah == 0) return;

    cubit.kirimSemuaPengajuan();
```

e. File navigation/app_routes

```dart
import 'package:flutter/material.dart';

import '../model/layanan.dart';
import '../pages/halaman_rincian_layanan.dart';
import 'kerangka_navigasi.dart';

class AppRoutes {
  AppRoutes._();

  static const String beranda = '/';
  static const String layanan = '/layanan';
  static const String warga = '/warga';
  static const String rincianLayanan = '/rincian-layanan';

  static Map<String, WidgetBuilder> daftarRoute() {
    return {
      beranda: (_) => const KerangkaNavigasi(
            initialIndex: 0,
          ),
      layanan: (_) => const KerangkaNavigasi(
            initialIndex: 1,
          ),
      warga: (_) => const KerangkaNavigasi(
            initialIndex: 2,
          ),
    };
  }

  static Route<dynamic>? bentukRoute(
    RouteSettings settings,
  ) {
    if (settings.name == rincianLayanan) {
      final arguments = settings.arguments;

      if (arguments is Layanan) {
        return MaterialPageRoute(
          builder: (_) => HalamanRincianLayanan(
            layanan: arguments,
          ), // HalamanRincianLayanan
```

f. File navigation/kerangka_navigation

```dart
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubit/pengajuan_cubit.dart';
import '../pages/halaman_beranda.dart';
import '../pages/halaman_layanan.dart';
import '../pages/halaman_warga.dart';

class KerangkaNavigasi extends StatefulWidget {
  final int initialIndex;

  const KerangkaNavigasi({
    super.key,
    this.initialIndex = 0,
  });

  @override
  State<KerangkaNavigasi> createState() =>
      _KerangkaNavigasiState();
}

class _KerangkaNavigasiState
    extends State<KerangkaNavigasi> {
  late int _indeksTerpilih;

  final List<Widget> _halaman = const [
    HalamanBeranda(),
    HalamanLayanan(),
    HalamanWarga(),
  ];

  final List<String> _judul = const [
    'Nusantara Cerdas',
    'Daftar Layanan',
    'Layanan Warga',
  ];
```

g. File models/favorite_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

class FavoritCubit extends Cubit<Set<String>> {
  FavoritCubit() : super(<String>{});

  bool isFavorit(String namaLayanan) {
    return state.contains(namaLayanan);
  }

  void tandai(String namaLayanan) {
    if (namaLayanan.trim().isEmpty ||
        state.contains(namaLayanan)) {
      return;
    }

    final favoritBaru = Set<String>.from(state);
    favoritBaru.add(namaLayanan);

    emit(favoritBaru);
  }

  void batalTandai(String namaLayanan) {
    if (!state.contains(namaLayanan)) return;

    final favoritBaru = Set<String>.from(state);
    favoritBaru.remove(namaLayanan);

    emit(favoritBaru);
  }

  void ubahFavorit(String namaLayanan) {
    if (isFavorit(namaLayanan)) {
      batalTandai(namaLayanan);
    } else {
      tandai(namaLayanan);
    }
  }
}
```

k. File pengajuan_cubit.dart

```dart
import 'package:flutter_bloc/flutter_bloc.dart';

class PengajuanState {
  final List<String> keranjang;
  final List<String> riwayatPengajuan;

  PengajuanState({
    required List<String> keranjang,
    required List<String> riwayatPengajuan,
  })  : keranjang = List.unmodifiable(keranjang),
        riwayatPengajuan =
            List.unmodifiable(riwayatPengajuan);

  int get totalPengajuan => keranjang.length;

  int get totalTerkirim => riwayatPengajuan.length;

  bool sudahDiKeranjang(String namaLayanan) {
    return keranjang.contains(namaLayanan);
  }
}

class PengajuanCubit extends Cubit<PengajuanState> {
  PengajuanCubit()
      : super(
          PengajuanState(
            keranjang: [],
            riwayatPengajuan: [],
          ),
        );

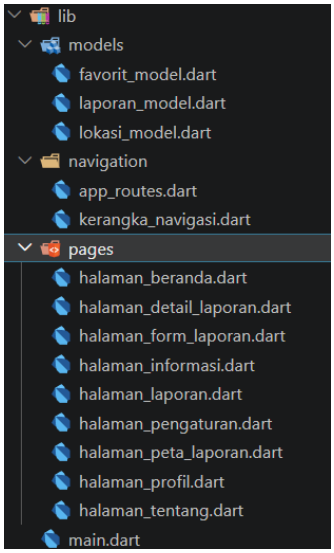
  void tambahPengajuan(String namaLayanan) {
    final nama = namaLayanan.trim();

    if (nama.isEmpty || state.sudahDiKeranjang(nama)) {
      return;
    }
```

4.2 Hasil Dokumentasi Program Tubes

Pada dasarnya file nya sama, namun ada sedikit perubahan dari praktikum sebelumnya seperti mengubah main, menambah folder models, mengubah kerangka navigation, mengubah halaman laporan, detail laporan, dan profil

a. Menyiapkan File lib (Halaman)



b. isi kode halaman utama

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'models/favorit_model.dart';
import 'navigation/app_routes.dart';

Run | Debug | Profile
void main() {
  runApp(const AplikasiSiagaManado());
}

class AplikasiSiagaManado extends StatelessWidget {
  const AplikasiSiagaManado({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(
          create: (context) => FavoritModel(),
        ), // ChangeNotifierProvider
      ],
      child: MaterialApp(
        title: 'Siaga Manado Mobile',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorSchemeSeed: Colors.red,
          useMaterial3: true,
        ), // ThemeData
        initialRoute: AppRoutes.beranda,
        routes: AppRoutes.daftarRoute(),
        onGenerateRoute: AppRoutes.bentukRoute,
        onUnknownRoute: AppRoutes.routeTidakDikenal,
      ), // MaterialApp
    ); // MultiProvider
  }
}
```

c. Isi Kode Halaman Laporan

```dart
import 'package:flutter/material.dart';
import '../navigation/app_routes.dart';

class HalamanLaporan extends StatelessWidget {
  const HalamanLaporan({super.key});

  @override
  Widget build(BuildContext context) {
    final List<Map<String, String>> laporan = [
      {
        'judul': 'Banjir di Kecamatan Malalayang',
        'lokasi': 'Kecamatan Malalayang',
        'status': 'Sedang Ditangani',
        'deskripsi':
            'Laporan genangan air yang mengganggu akses masyarakat di sekitar wilayah M
      },
      {
        'judul': 'Tanah Longsor di Kecamatan Bunaken',
        'lokasi': 'Kecamatan Bunaken',
        'status': 'Diverifikasi',
        'deskripsi':
            'Material tanah menutup sebagian akses jalan dan membutuhkan penanganan petu
      },
      {
        'judul': 'Pohon Tumbang di Kecamatan Sario',
        'lokasi': 'Kecamatan Sario',
        'status': 'Selesai',
        'deskripsi':
            'Pohon tumbang telah dievakuasi dan akses jalan telah kembali normal.',
      },
    ];

    return ListView.separated(
      padding: const EdgeInsets.all(16),
      itemCount: laporan.length,
      separatorBuilder: (context, index) {
        return const SizedBox(height: 12);
      },
```

c. Isi Kode Halaman detail laporan

```dart
import 'package:flutter/material.dart';

class HalamanDetailLaporan extends StatelessWidget {
  final String judul;
  final String lokasi;
  final String status;
  final String deskripsi;

  const HalamanDetailLaporan({
    super.key,
    required this.judul,
    required this.lokasi,
    required this.status,
    required this.deskripsi,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Detail Laporan'),
      ), // AppBar
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Icon(
              Icons.report,
              size: 64,
            ), // Icon
            const SizedBox(height: 20),
            Text(
              judul,
              style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                    fontWeight: FontWeight.bold,
                  ),
            ), // Text
```

d. File Halaman Informasi

```dart
import 'package:flutter/material.dart';

class HalamanInformasi extends StatelessWidget {
  const HalamanInformasi({super.key});

  @override
  Widget build(BuildContext context) {
    return DefaultTabController(
      length: 3,
      child: Column(
        children: [
          const TabBar(
            tabs: [
              Tab(
                icon: Icon(Icons.flood),
                text: 'Banjir',
              ), // Tab
              Tab(
                icon: Icon(Icons.landslide),
                text: 'Longsor',
              ), // Tab
              Tab(
                icon: Icon(Icons.warning),
                text: 'Lainnya',
              ), // Tab
            ],
          ), // TabBar
          Expanded(
            child: TabBarView(
              children: [
                _buatDaftar([
                  'Waspada genangan saat hujan deras.',
                  'Hindari melewati jalan yang tergenang.',
                  'Pantau informasi dari petugas terkait.',
                ]),
                _buatDaftar([
                  'Waspadai area dengan kemiringan tanah tinggi.',
                  'Hindari berada di bawah tebing ketika hujan.',
                  'Segera laporkan tanda-tanda pergerakan tanah.',
```

e. File Halaman Pengaturan

```dart
import 'package:flutter/material.dart';

class HalamanPengaturan extends StatelessWidget {
  const HalamanPengaturan({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Pengaturan'),
      ), // AppBar
      body: ListView(
        children: [
          SwitchListTile(
            value: true,
            onChanged: (value) {},
            title: const Text('Notifikasi'),
            subtitle: const Text(
              'Terima pemberitahuan mengenai informasi kebencanaan.',
            ), // Text
          ), // SwitchListTile
          const Divider(),
          ListTile(
            leading: const Icon(Icons.language),
            title: const Text('Bahasa'),
            subtitle: const Text('Bahasa Indonesia'),
          ), // ListTile
          ListTile(
            leading: const Icon(Icons.location_on),
            title: const Text('Lokasi'),
            subtitle: const Text('Kota Manado'),
          ), // ListTile
        ],
      ), // ListView
    ); // Scaffold
  }
}
```

f. File Halaman Profil

```dart
import 'package:flutter/material.dart';
import '../navigation/app_routes.dart';

class HalamanProfil extends StatelessWidget {
  const HalamanProfil({super.key});

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(
      padding: const EdgeInsets.all(20),
      child: Column(
        children: [
          const CircleAvatar(
            radius: 50,
            child: Icon(
              Icons.person,
              size: 50,
            ), // Icon
          ), // CircleAvatar
          const SizedBox(height: 16),
          Text(
            'Warga Kota Manado',
            style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                  fontWeight: FontWeight.bold,
                ),
          ), // Text
          const SizedBox(height: 8),
          const Text(
            'Pengguna aplikasi Siaga Manado Mobile',
          ), // Text
          const SizedBox(height: 24),
          Card(
            child: Column(
              children: [
                const ListTile(
                  leading: Icon(Icons.badge),
                  title: Text('Identitas Pengguna'),
```

g. File Halaman Profil

```dart
import 'package:flutter/material.dart';

class HalamanTentang extends StatelessWidget {
  const HalamanTentang({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tentang Aplikasi'),
      ), // AppBar
      body: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const CircleAvatar(
                radius: 45,
                child: Icon(
                  Icons.shield,
                  size: 50,
                ), // Icon
              ), // CircleAvatar
              const SizedBox(height: 20),
              Text(
                'Siaga Manado Mobile',
                style: Theme.of(context).textTheme.headlineSmall?.copyWith(
                      fontWeight: FontWeight.bold,
                    ),
                textAlign: TextAlign.center,
              ), // Text
              const SizedBox(height: 12),
              const Text(
                'Aplikasi purwarupa layanan informasi dan pelaporan kebencanaan Kota Mar
                textAlign: TextAlign.center,
              ), // Text
              const SizedBox(height: 20),
```

h. File app routes

```dart
import '../pages/halaman_detail_laporan.dart';
import '../pages/halaman_informasi.dart';
import '../pages/halaman_laporan.dart';
import '../pages/halaman_pengaturan.dart';
import '../pages/halaman_profil.dart';
import '../pages/halaman_tentang.dart';
import 'kerangka_navigasi.dart';

class AppRoutes {
  static const String beranda = '/';
  static const String laporan = '/laporan';
  static const String informasi = '/informasi';
  static const String profil = '/pr
  static const String detailLaporan
  static const String pengaturan = '/pengaturan';
  static const String tentang = '/tentang';

  static Map<String, WidgetBuilder> daftarRoute() {
    return {
      beranda: (context) => const KerangkaNavigasi(),
      pengaturan: (context) => const HalamanPengaturan(),
      tentang: (context) => const HalamanTentang(),
    };
  }

  static Route<dynamic>? bentukRoute(RouteSettings settings) {
    switch (settings.name) {
      case laporan:
        return MaterialPageRoute(
          builder: (context) => const HalamanLaporan(),
          settings: settings,
        ); // MaterialPageRoute
```

Type: String

an';

i. FIle kerangka_navigation

```dart
import 'package:flutter/material.dart';
import '../pages/halaman_beranda.dart';
import '../pages/halaman_informasi.dart';
import '../pages/halaman_laporan.dart';
import '../pages/halaman_profil.dart';
import 'app_routes.dart';

class KerangkaNavigasi extends StatefulWidget {
  const KerangkaNavigasi({super.key});

  @override
  State<KerangkaNavigasi> createState() => _KerangkaNavigasiState();
}

class _KerangkaNavigasiState extends State<KerangkaNavigasi> {
  int _indeksTerpilih = 0;

  final List<Widget> _halaman = [
    const HalamanBeranda(),
    const HalamanLaporan(),
    const HalamanInformasi(),
    const HalamanProfil(),
  ];

  final List<String> _judul = [
    'Beranda',
    'Laporan',
    'Informasi',
    'Profil',
  ];

  void _pilihTujuan(int indeks) {
    setState(() {
      _indeksTerpilih = indeks;
    });
  }
```

j. File models/favorit_model.dart

```dart
import 'package:flutter/foundation.dart';

class FavoritModel extends ChangeNotifier {
  final List<String> _daftarFavorit = [];

  List<String> get daftarFavorit {
    return List.unmodifiable(_daftarFavorit);
  }

  int get totalFavorit {
    return _daftarFavorit.length;
  }

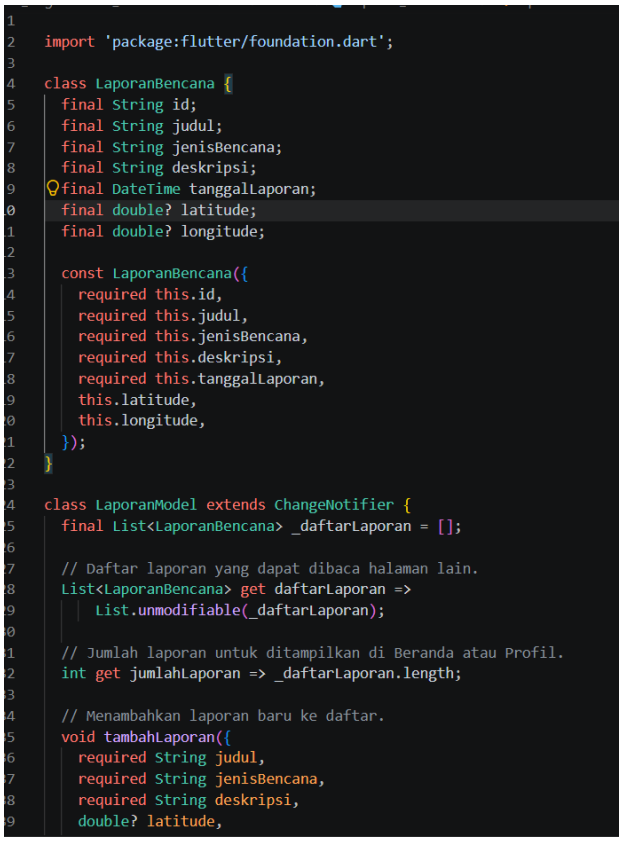
  bool isFavorit(String namaLaporan) {
    return _daftarFavorit.contains(namaLaporan);
  }

  void tambahFavorit(String namaLaporan) {
    if (!_daftarFavorit.contains(namaLaporan)) {
      _daftarFavorit.add(namaLaporan);
      notifyListeners();
    }
  }

  void hapusFavorit(String namaLaporan) {
    if (_daftarFavorit.contains(namaLaporan)) {
      _daftarFavorit.remove(namaLaporan);
      notifyListeners();
    }
  }

  void ubahFavorit(String namaLaporan) {
    if (isFavorit(namaLaporan)) {
      hapusFavorit(namaLaporan);
    } else {
      tambahFavorit(namaLaporan);
    }
  }
}
```

h. Isi Halaman laporan_model



```dart

import 'package:flutter/foundation.dart';

class LaporanBencana {
  final String id;
  final String judul;
  final String jenisBencana;
  final String deskripsi;
  final DateTime tanggalLaporan;
  final double? latitude;
  final double? longitude;

  const LaporanBencana({
    required this.id,
    required this.judul,
    required this.jenisBencana,
    required this.deskripsi,
    required this.tanggalLaporan,
    this.latitude,
    this.longitude,
  });
}

class LaporanModel extends ChangeNotifier {
  final List<LaporanBencana> _daftarLaporan = [];

  // Daftar laporan yang dapat dibaca halaman lain.
  List<LaporanBencana> get daftarLaporan =>
      List.unmodifiable(_daftarLaporan);

  // Jumlah laporan untuk ditampilkan di Beranda atau Profil.
  int get jumlahLaporan => _daftarLaporan.length;

  // Menambahkan laporan baru ke daftar.
  void tambahLaporan({
    required String judul,
    required String jenisBencana,
    required String deskripsi,
    double? latitude,
```

i. Isi Halaman lokasi_model

```dart
import 'package:flutter/foundation.dart';

class LokasiModel extends ChangeNotifier {
  // Koordinat Kota Manado sebagai posisi awal kamera peta.
  static const double latitudeAwal = 1.4748;
  static const double longitudeAwal = 124.8421;

  double? _latitude;
  double? _longitude;
  String? _namaLokasi;

  // Getter agar halaman lain dapat membaca lokasi pilihan.
  double? get latitude => _latitude;
  double? get longitude => _longitude;
  String? get namaLokasi => _namaLokasi;

  // Mengecek apakah pengguna sudah memilih titik.
  bool get sudahMemilihLokasi =>
      _latitude != null && _longitude != null;

  // Digunakan sebagai pusat awal kamera Google Maps.
  double get pusatLatitude => _latitude ?? latitudeAwal;
  double get pusatLongitude => _longitude ?? longitudeAwal;

  // Menyimpan koordinat yang dipilih pengguna di peta.
  void pilihLokasi({
    required double latitude,
    required double longitude,
    String? namaLokasi,
  }) {
    _latitude = latitude;
    _longitude = longitude;
    _namaLokasi = namaLokasi;

    notifyListeners();
  }
```

Isi Halaman Form laporan

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/laporan_model.dart';
import '../models/lokasi_model.dart';
import '../navigation/app_routes.dart';

class HalamanFormLaporan extends StatefulWidget {
  const HalamanFormLaporan({super.key});

  @override
  State<HalamanFormLaporan> createState() =>
      _HalamanFormLaporanState();
}

class _HalamanFormLaporanState
    extends State<HalamanFormLaporan> {
  final _formKey = GlobalKey<FormState>();

  final _judulController = TextEditingController();
  final _deskripsiController = TextEditingController();

  String? _jenisBencana;
  bool _konfirmasiKebenaran = false;
  bool _sedangMengirim = false;

  @override
  void dispose() {
    _judulController.dispose();
    _deskripsiController.dispose();
    super.dispose();
  }

  Future<void> _pilihLokasi() async {
    await Navigator.pushNamed(
      context,
      AppRoutes.petaLaporan,
    );
```

5. Analisis dan Pembahasan

1. Alasan Pemilihan Stateless Widget dan Stateful Widget

Pada praktikum Modul 3, aplikasi dikembangkan dengan menambahkan pengelolaan state dan interaksi pengguna agar perubahan data dapat ditampilkan secara dinamis pada antarmuka. Pemilihan jenis widget disesuaikan dengan kebutuhan masing-masing bagian aplikasi. StatelessWidget digunakan pada bagian yang tidak memiliki perubahan state lokal, seperti halaman yang hanya menampilkan informasi atau struktur antarmuka yang bersifat tetap. Sementara itu, StatefulWidget digunakan pada halaman detail laporan karena terdapat status proses yang berubah selama pengguna berinteraksi dengan aplikasi, yaitu kondisi saat laporan sedang diproses. Perubahan status tersebut dikelola menggunakan setState() sehingga antarmuka dapat menampilkan indikator proses secara langsung.

Untuk data yang hanya diperlukan pada satu halaman, penggunaan setState() dinilai sesuai karena state tidak perlu diakses oleh halaman lain. Pada halaman detail laporan, variabel _isMemuat digunakan untuk menentukan apakah proses penanganan laporan sedang berlangsung. Ketika pengguna menekan tombol tindakan, nilai tersebut diubah melalui setState() sehingga tombol berubah menjadi kondisi nonaktif dan menampilkan indikator CircularProgressIndicator. Setelah proses selesai, state dikembalikan dan halaman ditutup menggunakan Navigator.pop() dengan mengirimkan pesan hasil kepada halaman sebelumnya.

Berbeda dengan state lokal, data favorit digunakan pada lebih dari satu halaman sehingga dikelola menggunakan ChangeNotifier dan Provider. FavoritModel menyimpan daftar laporan yang ditandai sebagai favorit dan menyediakan method untuk menambah, menghapus, serta mengubah status favorit. notifyListeners() dipanggil setelah data berubah agar widget yang menggunakan Provider dapat memperbarui tampilannya secara otomatis. context.read() digunakan ketika aplikasi hanya perlu menjalankan perubahan data, sedangkan Consumer atau context.watch() digunakan ketika bagian antarmuka harus bereaksi terhadap perubahan data. Selain itu, context.select() digunakan pada badge jumlah favorit agar hanya bagian yang membutuhkan nilai jumlah favorit yang diperbarui ketika state berubah.

2. Kendala dan Penyelesaian

Salah satu kendala dalam penerapan state management adalah memastikan Provider dapat ditemukan oleh seluruh halaman yang membutuhkan data bersama. Jika FavoritModel ditempatkan pada tingkat widget yang terlalu rendah, halaman lain dapat mengalami ProviderNotFoundException. Permasalahan tersebut diselesaikan dengan menempatkan MultiProvider pada akar aplikasi, sebelum MaterialApp, sehingga halaman yang diakses melalui named route maupun kerangka navigasi dapat memperoleh FavoritModel yang sama. Dengan demikian, perubahan favorit pada halaman Laporan dapat langsung terlihat pada halaman Profil tanpa perlu mengirimkan data secara manual antarhalaman.

Kendala lain berkaitan dengan perubahan state setelah proses asynchronous. Pada halaman detail laporan, proses simulasi menggunakan Future.delayed() sehingga terdapat jeda sebelum state dikembalikan dan hasil dikirim ke halaman sebelumnya. Untuk mencegah pemanggilan setState() ketika widget sudah tidak aktif, dilakukan pemeriksaan mounted sebelum memperbarui state setelah proses asynchronous selesai. Hal ini membuat pengelolaan state lebih aman ketika halaman telah ditutup sebelum proses selesai.

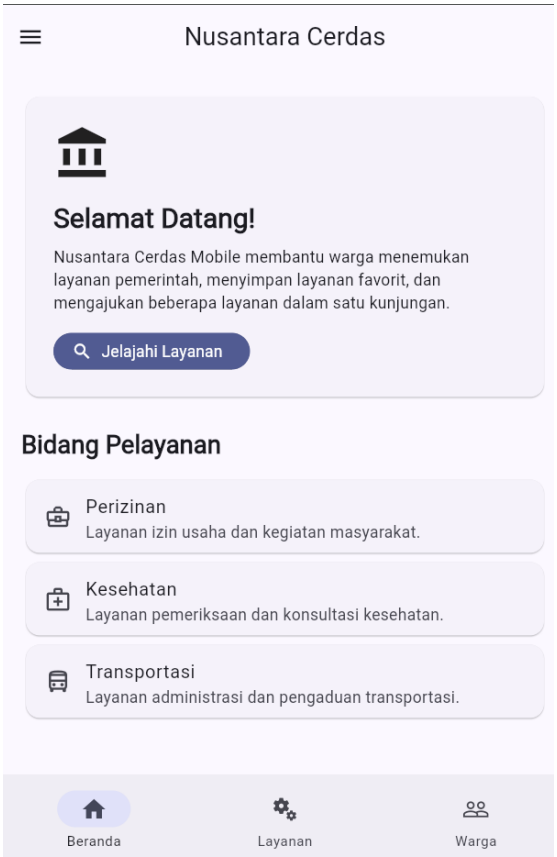
3. Keterkaitan dengan Pilar Smart City

Aplikasi Siaga Manado memiliki keterkaitan dengan konsep Smart City, khususnya pada aspek Smart Governance dan Smart Living. Pada aspek Smart Governance, aplikasi menyediakan sarana digital untuk menyampaikan dan memantau laporan masyarakat sehingga informasi mengenai kondisi wilayah dapat

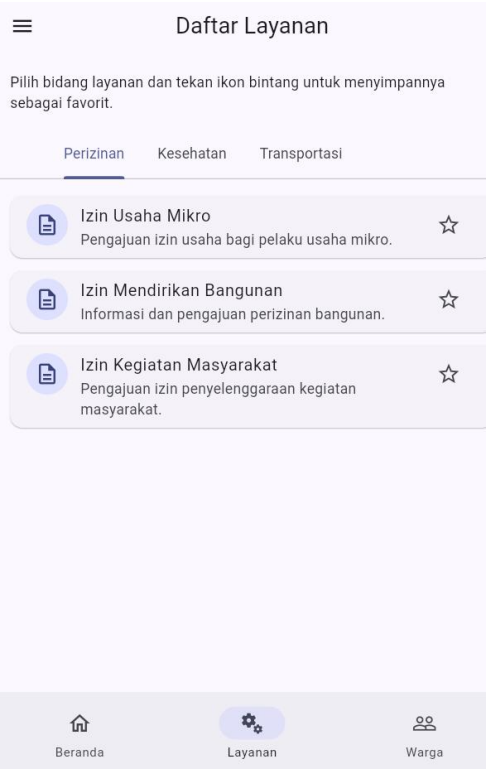
dikelola secara lebih terstruktur. Sementara itu, dari sisi Smart Living, penyediaan informasi laporan dan status penanganannya dapat membantu masyarakat memperoleh informasi mengenai kondisi lingkungan dan layanan kebencanaan di sekitarnya. Dengan adanya state management, interaksi tersebut dapat direpresentasikan secara lebih dinamis sehingga purwarupa aplikasi tidak hanya menampilkan informasi, tetapi juga memberikan respons terhadap aktivitas pengguna.

6. Tampilan Halaman

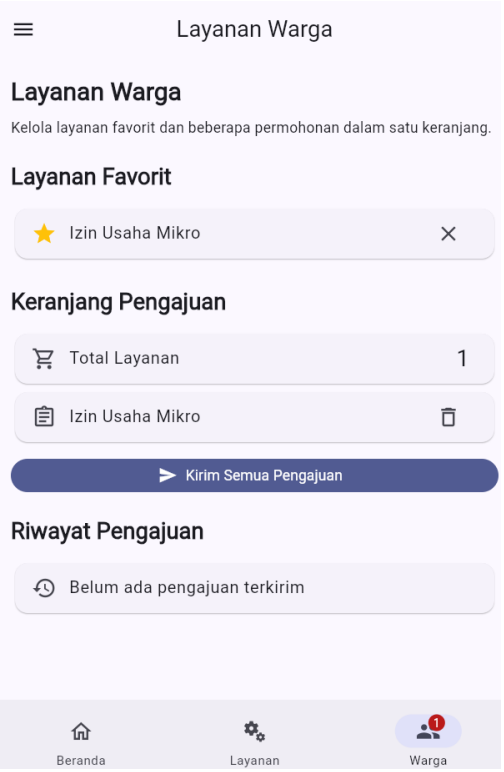
a. Halaman Nusantara Cerdas



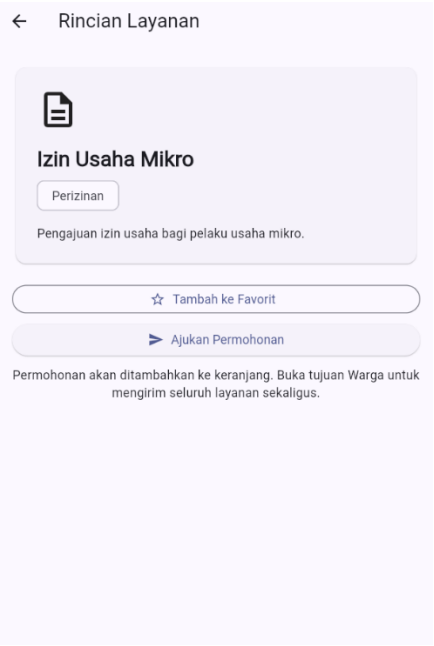
Halaman Beranda menampilkan informasi utama aplikasi Nusantara Cerdas Mobile serta tiga bidang layanan, yaitu Perizinan, Kesehatan, dan Transportasi. Halaman ini menjadi titik awal bagi warga untuk mengakses layanan yang tersedia.



Halaman Layanan menampilkan daftar layanan berdasarkan bidang melalui TabBar. Setiap layanan memiliki ikon bintang yang dapat digunakan untuk menandai layanan sebagai favorit.



Halaman Warga menampilkan layanan favorit yang sebelumnya ditandai dari berbagai tab bidang layanan. Penggunaan BlocBuilder memungkinkan daftar favorit diperbarui secara otomatis ketika terjadi perubahan state.

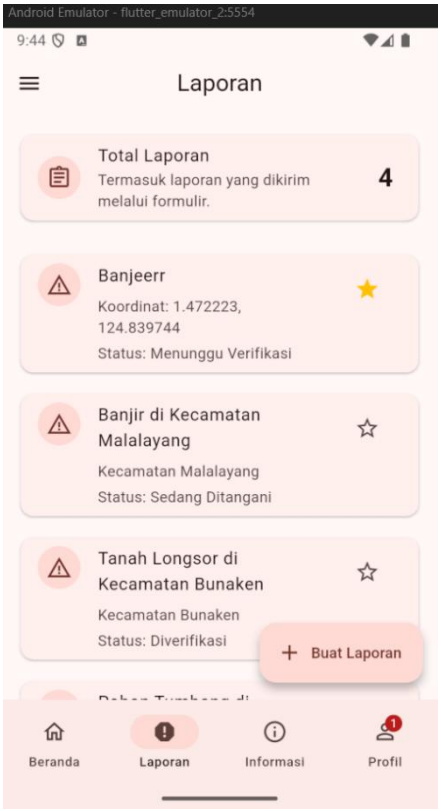


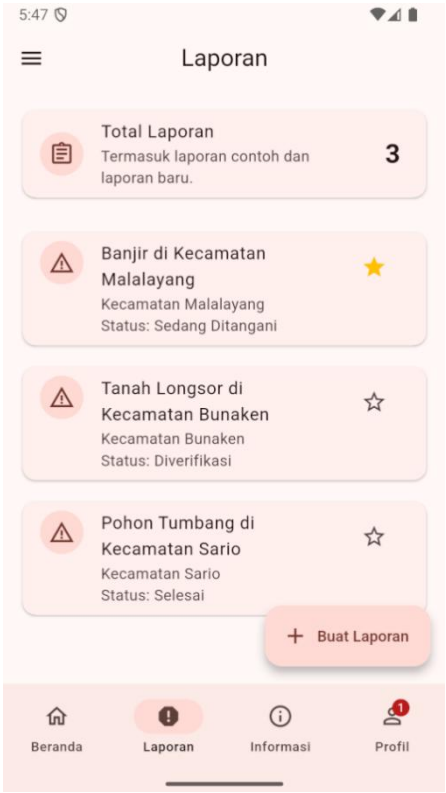
Halaman Rincian Layanan menampilkan nama, kategori, dan deskripsi layanan yang dipilih. Halaman ini juga menyediakan tombol untuk mengelola favorit dan menambahkan layanan ke keranjang pengajuan.

b. Siaga Manado(tugas akhir)

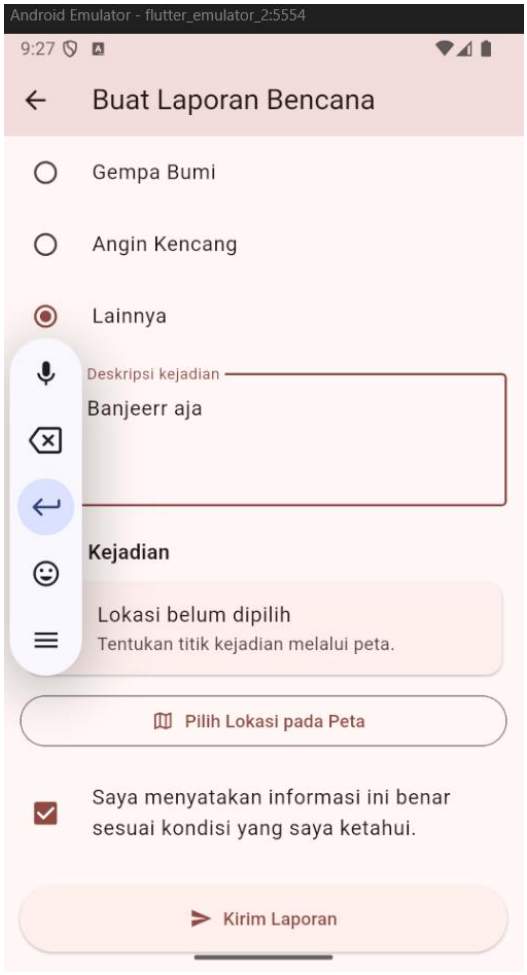


Halaman diatas merupakan halaman tugas besar siaga manado, pelaporan bencana alam, terdapat navbar laporan, informasi, dan profil

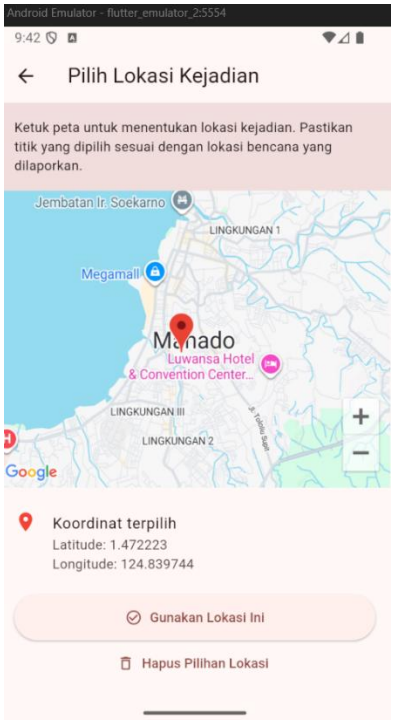




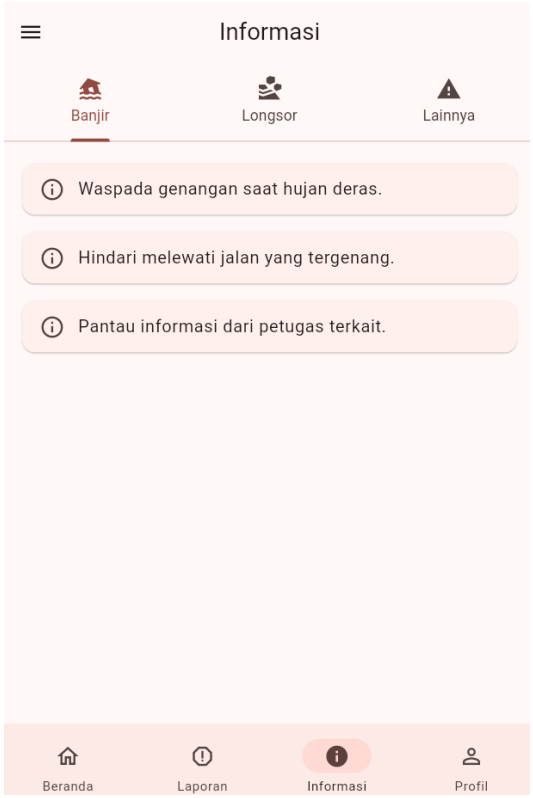
Halaman diatas merupakan halaman laporan yang dimana user bisa melihat laporan, menambah favorit sehingga bisa dilihat di profil nantinya. User juga bisa menambahkan laporan dan menuju form laporan dan menekan sesuai lokasinya



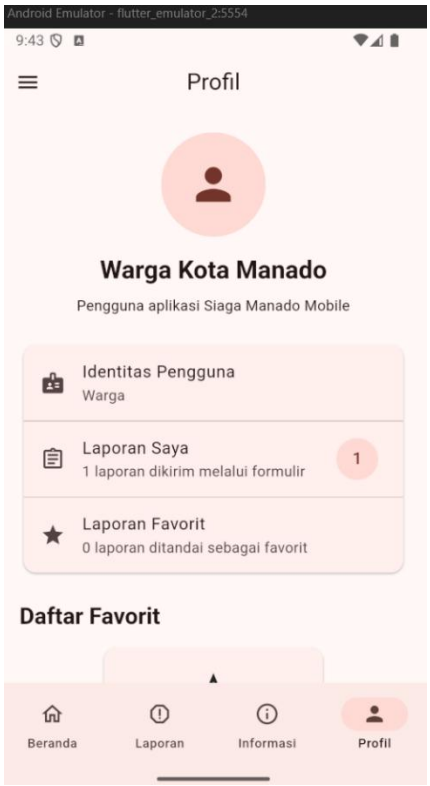
Isi Form Laporan bencana alam



Penggunaan maps/peta menggunakan API gmaps



Halaman diatas merupakan halaman informasi / mitigasi seperti banjir, longsor, dan lainnya



Dan yang terakhir merupakan halaman profil dimana user bisa melihat laporannya, identitasnya hingga bisa melihat daftar favorit, dimana kita membuat daftar berita favorit dari halaman laporan sebelumnya akan terlihat di profil.

7. Kesimpulan

Pada praktikum Modul 5, aplikasi Siaga Manado dikembangkan dengan mengganti pendekatan *state management* dari ChangeNotifier dan Provider menjadi Cubit menggunakan pustaka flutter_bloc. Pengelolaan state bersama diterapkan melalui LaporanCubit untuk menyimpan data laporan bencana dan FavoritCubit untuk mengelola daftar laporan favorit. Kedua Cubit disediakan menggunakan MultiBlocProvider pada akar aplikasi sehingga dapat diakses oleh beberapa halaman secara konsisten.
Penggunaan BlocBuilder memungkinkan halaman Beranda, Laporan, dan Profil menampilkan perubahan data secara otomatis. Sementara itu, BlocSelector digunakan pada ikon favorit, badge navigasi, dan jumlah laporan agar pembaruan tampilan lebih terfokus. Method context.read() digunakan untuk menjalankan perubahan state, seperti menambahkan laporan dan mengubah status favorit.
Adapun setState() tetap digunakan untuk state lokal yang hanya diperlukan pada halaman tertentu, seperti status sedang mengirim pada formulir, pemilihan jenis bencana, dan penentuan titik lokasi pada Google Maps. Pemilihan Cubit didasarkan pada kebutuhan aplikasi yang relatif sederhana karena perubahan state dapat dikelola melalui method langsung tanpa memerlukan kelas Event seperti pada BLoC.
Dengan demikian, penerapan Cubit memungkinkan aplikasi Siaga Manado mengelola data bersama secara reaktif, mempertahankan navigasi sebelumnya, serta memisahkan state lokal dan state aplikasi sesuai tanggung jawab masing-masing komponen.

8. Github

https://github.com/Muhammad-Haikal-MH/PEMROGRAMAN-PERANGKAT-BERGERAK-LANJUT-Mobile-Apps-